# 5. Plano de Garantia da Qualidade (PGQ)

## 5.1 Padrões de codificação

- Estilo PEP 8, verificado pelo ruff, com linhas de até 100 caracteres. Código com aviso do ruff não é mesclado.
- Nomes em português: classes em PascalCase (ColaboradorPadrao); funções, métodos e variáveis em snake_case (calcular_salario_bruto); constantes em maiúsculas.
- Anotação de tipo em parâmetros e retornos.
- Docstring em toda classe e método público, citando o requisito ou a regra que implementa (exemplo: "RN003: salário base não pode ser negativo.").
- Violação de regra de negócio lança ValidacaoError com o código da regra na mensagem (exemplo: "RN001: …").
- A camada de domínio não imprime nada nem acessa banco; impressão e leitura de tela ficam só nas interfaces.
- Valores monetários são exibidos com duas casas decimais.
- Nenhuma exceção é silenciada: todo except informa o que falhou.

## 5.2 Controle de versões

- Repositório Git no GitHub: johnrosik/folha-pagamento-gqs, com a estrutura de entrega de 01 a 11.
- O ramo main guarda sempre uma versão que executa. Ninguém trabalha direto nele.
- Cada tarefa é feita num ramo próprio, nomeado pelo tipo e pela issue: feat/12-cadastro-sqlite, fix/30-irrf-negativo, docs/9-readme.
- Mensagens de commit em português, no imperativo, com o número da issue: "Valida matrícula duplicada no banco (#12)".
- Toda mudança entra por Pull Request, revisado e aprovado por outro integrante antes da mesclagem. A mesclagem preserva os commits, para o histórico mostrar o trabalho de cada um.
- Tarefas, defeitos e cronograma ficam nas Issues e no GitHub Projects; todo Pull Request cita a issue que resolve.

## 5.3 Processo de revisão técnica

**Inspeção de código.** Certificar que o código executa todos os requisitos e regras; verificar possíveis erros, bugs e outros problemas; garantir que o front-end está conectado ao back-end, sem função sem representação visual.

**Artefatos e software.** Analisar a qualidade da documentação técnica e dos modelos conceituais; revisar a correspondência entre o código-fonte e os diagramas UML; o software precisa estar de acordo com a definição técnica do projeto, seguindo as regras e a organização definidas pela equipe.

**Registro e encerramento.** Toda não conformidade, no código ou nos documentos e modelos, vira uma issue no repositório: etiqueta bug para código e documentação para artefatos, com severidade (crítica, alta, média ou baixa), responsável e prazo. A issue só é fechada quando a correção entra por Pull Request aprovado por outro integrante e, no caso de código, os testes automatizados passam. Cada revisão gera uma ata (modelo no Anexo) com data, participantes, artefatos revisados, não conformidades, correções e responsáveis, citando as issues abertas.

## 5.4 Critérios de qualidade

Os critérios de qualidade deste projeto seguem a norma ISO/IEC 25010.

| Característica | Critério no projeto | Como se verifica |
|---|---|---|
| **Adequação funcional** | Todos os requisitos funcionais executados e os cálculos de salário, descontos e totais sem erro matemático. | CT001 a CT024 aprovados, com os valores conferidos centavo a centavo contra o resultado esperado. |
| **Eficiência de desempenho** | Cadastro, consulta, folha individual e resumo respondem em até 1 segundo com 100 colaboradores cadastrados. | Teste de sistema com 100 colaboradores fictícios; o tempo de cada operação é medido no teste automatizado e registrado no Relatório de Testes. |
| **Compatibilidade** | Executa com Python 3.12 no Windows 10/11 e no Linux (Ubuntu 22.04); a página web funciona no Chrome e no Edge atuais, sem bibliotecas além das listadas em 4.2. | CT019 e CT020 executados em cada combinação de sistema e navegador, com o resultado anotado no Relatório de Testes. |
| **Usabilidade** | Uma pessoa que não participou do projeto cadastra um colaborador e emite o resumo sem ajuda, em até 5 minutos; toda mensagem de erro cita o campo e a regra violada. | CT024, executado por uma pessoa de fora da equipe; mensagens conferidas nos CT010 a CT016. |
| **Confiabilidade** | Dado inválido não encerra a aplicação; os registros gravados sobrevivem ao fechamento do sistema. | CT010 a CT016 (erro tratado sem encerrar) e CT020 (dados restaurados). |
| **Segurança** | Consultas ao banco só com parâmetros; nenhum dado inválido é gravado. | Revisão de código (5.3) e testes de integração com banco temporário. |
| **Manutenibilidade** | Novo tipo de remuneração entra sem alterar os cálculos existentes (RNF004); código sem avisos do ruff. | CT023 e ruff sem avisos a cada Pull Request. |
| **Portabilidade** | O sistema roda copiando a pasta do projeto e instalando as dependências, sem configurar a máquina. | Instalação do zero numa máquina da equipe, seguindo o Manual Técnico. |

## 5.5 Métricas adotadas

| Métrica | Como se mede | Meta | Ferramenta |
|---|---|---|---|
| **Cobertura de testes** | linhas executadas pelos testes ÷ linhas do código | ≥ 80% | pytest-cov |
| **Requisitos atendidos** | RF com caso de teste aprovado ÷ total de RF | 100% | matriz de rastreabilidade |

| Métrica | Como se mede | Meta | Ferramenta |
|---|---|---|---|
| **Casos de teste aprovados** | aprovados ÷ executados | 100% na homologação | relatório de testes |
| **Defeitos críticos abertos** | issues bug com severidade crítica e abertas | 0 | GitHub Issues |
| **Taxa de correção** | defeitos corrigidos ÷ defeitos encontrados | 100% dos críticos e altos | GitHub Issues |
| **Densidade de defeitos** | defeitos encontrados ÷ mil linhas de código | acompanhamento | GitHub Issues |
| **Conformidade de estilo** | avisos do ruff | 0 | ruff |

As métricas são medidas a cada Pull Request (cobertura e estilo) e consolidadas no Relatório Final de Qualidade.

## 5.6 Critérios de aprovação

| ID | Item de avaliação | Critério para aprovação | Peso |
|---|---|---|---|
| **CA001** | Requisitos | 100% dos requisitos funcionais implementados e operacionais. | Obrigatório |
| **CA002** | Regras de negócio | Aprovação em todos os cenários de teste. | Obrigatório |
| **CA003** | Acurácia | O relatório exibe com precisão os cálculos dos salários. | Obrigatório |
| **CA004** | Persistência dos dados | Salvamento e restauração dos dados com sucesso. | Obrigatório |
| **CA005** | Qualidade da arquitetura | Aplicação total dos diagramas UML. | Importante |

## 5.7 Fluxo da garantia da qualidade

| 1. Requisitos | 2. Análise | 3. Desenvolvimento | 4. Testes | 5. Homologação | 6. Entrega |
|---|---|---|---|---|---|
| Visão e ERS revisadas | UML e arquitetura revisadas | Código por PR revisado | Unitários, integração e aceitação | Termo de homologação | Pastas 01 a 11 |

Cada etapa só começa quando a anterior passou pela revisão técnica descrita em 5.3.

# Anexo · Ata de revisão técnica

Modelo do Anexo I (seção 15), preenchido na revisão feita a partir da avaliação da A1 recebida em 21/10/2026.

| Item | Informação |
|---|---|
| **Projeto** | Financy · Sistema de Folha de Pagamento |
| **Data** | 30/10/2026 |
| **Participantes** | João Vitor Rosiak de O. Baltezan<br>Iago Ferst Tubiana |
| **Artefatos revisados** | Documento da A1 versão 1.0 (seções 1 a 9), diagramas UML 3.1 a 3.4 e organização do repositório |
| **Não conformidades encontradas** | 1. Remuneração por produção sem o salário base em RF004, UML e casos de teste.<br>2. Requisitos funcionais só com título; RF005 sem a listagem cadastral e remuneratória.<br>3. Matriz sem RNF001, RNF002 e RNF004.<br>4. Diagrama de classes sem alteração e sem o repositório; atividades sem a conversão do percentual.<br>5. Critérios de qualidade sem limites verificáveis; revisão sem registro e encerramento das não conformidades.<br>6. Casos de teste genéricos (CT006), sem alteração (CT009) e sem extensibilidade; CT017 descrito como no limite.<br>7. Artefatos concentrados em 11_Apresentacao; ata em branco. |
| **Correções combinadas** | Todas aplicadas na versão 1.1: fórmula da produção corrigida; RF com descrição; matriz com RNF001 a RNF004; diagramas de classes, sequência e atividades refeitos; critérios de qualidade com forma de verificação; registro e encerramento em 5.3; CT001 a CT024 com dados e valores; um PDF por pasta; esta ata. |
| **Responsável e prazo** | João: documento, diagramas e repositório (30/10, concluído). Iago: Google Doc da equipe (30/10, concluído). Pendente para a A3: o código herdado do T1 calcula a produção sem o salário base (05_Codigo_Fonte/folhadepagamento.py); João corrige na implementação, até 01/11. |